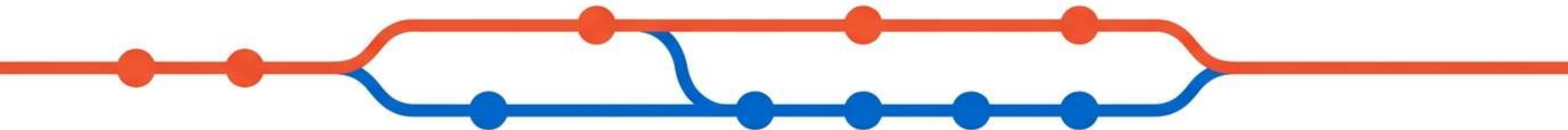


Git & GitHub: O Guia Definitivo

De projetos escolares confusos a um
fluxo de trabalho profissional.



0 PROBLEMA



Sobrescrever o trabalho do amigo sem querer.



Perder código importante.

Trabalho de Física



trabalho.doc



trabalho_final.doc



trabalho_final_FINAL_bruno.doc



trabalho_socorro_deu_erro.doc

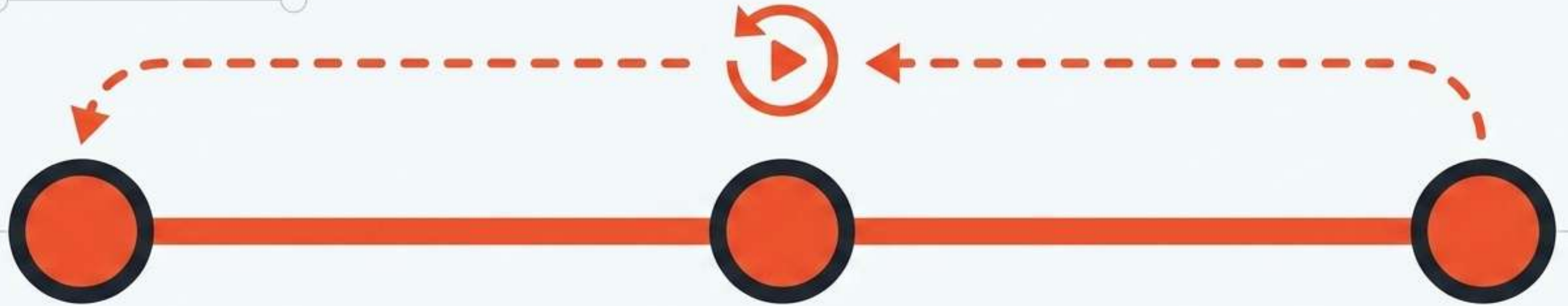


Medo constante de quebrar o projeto.



Nenhum histórico de quem fez o quê.

A Solução: Versionamento (Sua Máquina do Tempo)



Rastreamento Completo:

O poder de rastrear e gerenciar cada alteração no projeto.

Experimentação Segura:

Testar ideias novas sem comprometer o código original.

Recuperação (Rollback):

Reverter mudanças de maneira simples e instantânea se um bug crítico aparecer.

Uma Breve História (A Revolução de 2005)



Anos 1980 (RCS)

Local, focado em arquivos individuais. Lento e difícil de escalar.



Anos 1990 (Centralizado)

Servidor único. Trabalho colaborativo sujeito a gargalos e extrema lentidão.



2005 (A Revolução Git)

Linus Torvalds (criador do Linux) precisava de um sistema rápido para milhares de desenvolvedores. Construiu o Git em semanas.

Hoje, 94% dos desenvolvedores no mundo usam Git.

Git vs. GitHub: Qual é a Diferença?



Git



GitHub

O Motor (Gera a linha do tempo)

Metáfora

A Nuvem (Compartilha a linha do tempo)

Local (Instalado no seu PC)

Localização

Remoto (Na internet)

Linha de Comando (Terminal)

Interface

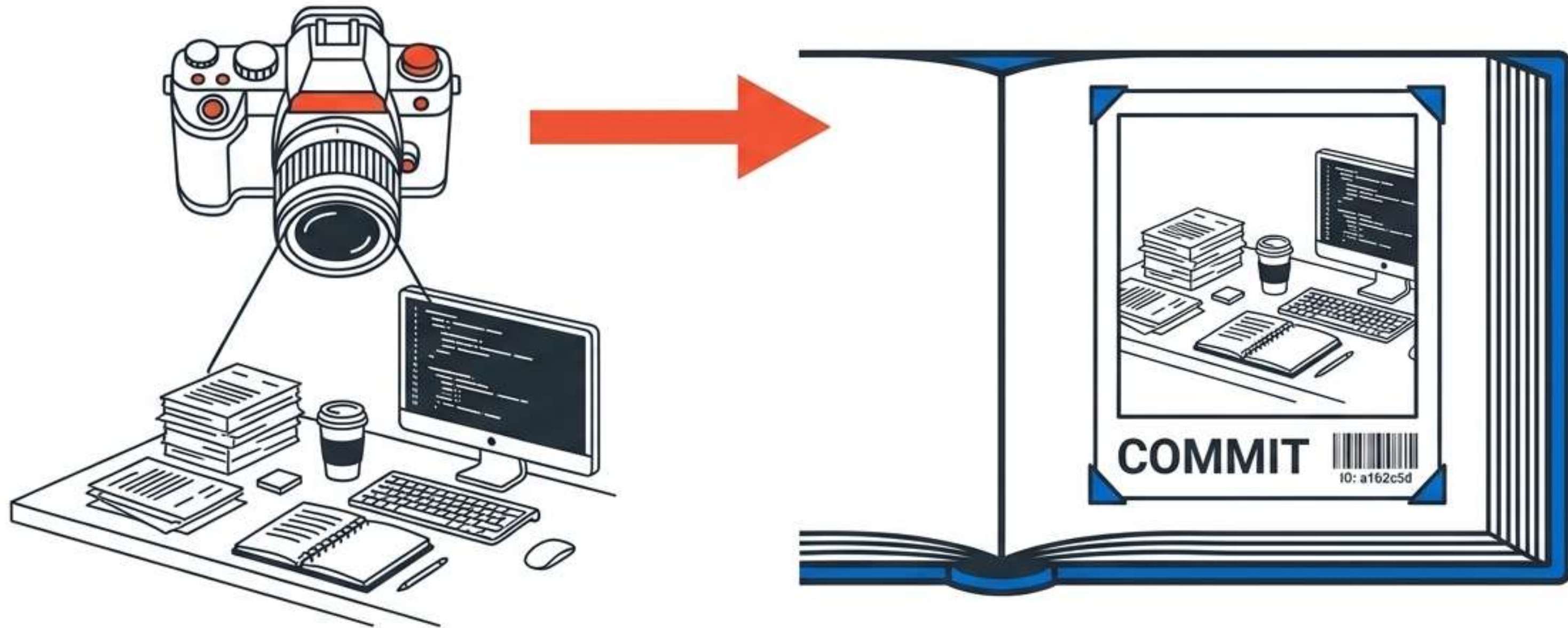
Interface Visual / Rede Social

Roda os seus projetos

Escala

Hospeda mais de 100 milhões
de repositórios globais

O Coração do Git: Commits (Snapshots)



- O Git não salva cópias inteiras dos arquivos a cada salvamento.
- Ele tira uma **Fotografia (Snapshot)** das diferenças entre as versões.
- No Git, essa fotografia é chamada de **Commit**.
- Cada Commit possui um identificador único à prova de falhas (Hash SHA-1), o nome do autor e a data.

Os 3 Estágios do Git

1. Working Directory



Onde você **edita**, **remove** ou **atualiza** seus arquivos no dia a dia.

git add

2. Staging Area



A **área de preparação**.
Você escolhe quais alterações vão entrar na próxima fotografia.

git commit

3. Repository



O **histórico oficial**.
A fotografia é confirmada e guardada no cofre.

Primeiros Passos: Instalação e Identidade

Instalação

Baixe em: `git-scm.com`

Verifique se funcionou no terminal:

```
> git --version
```

Configuração Global

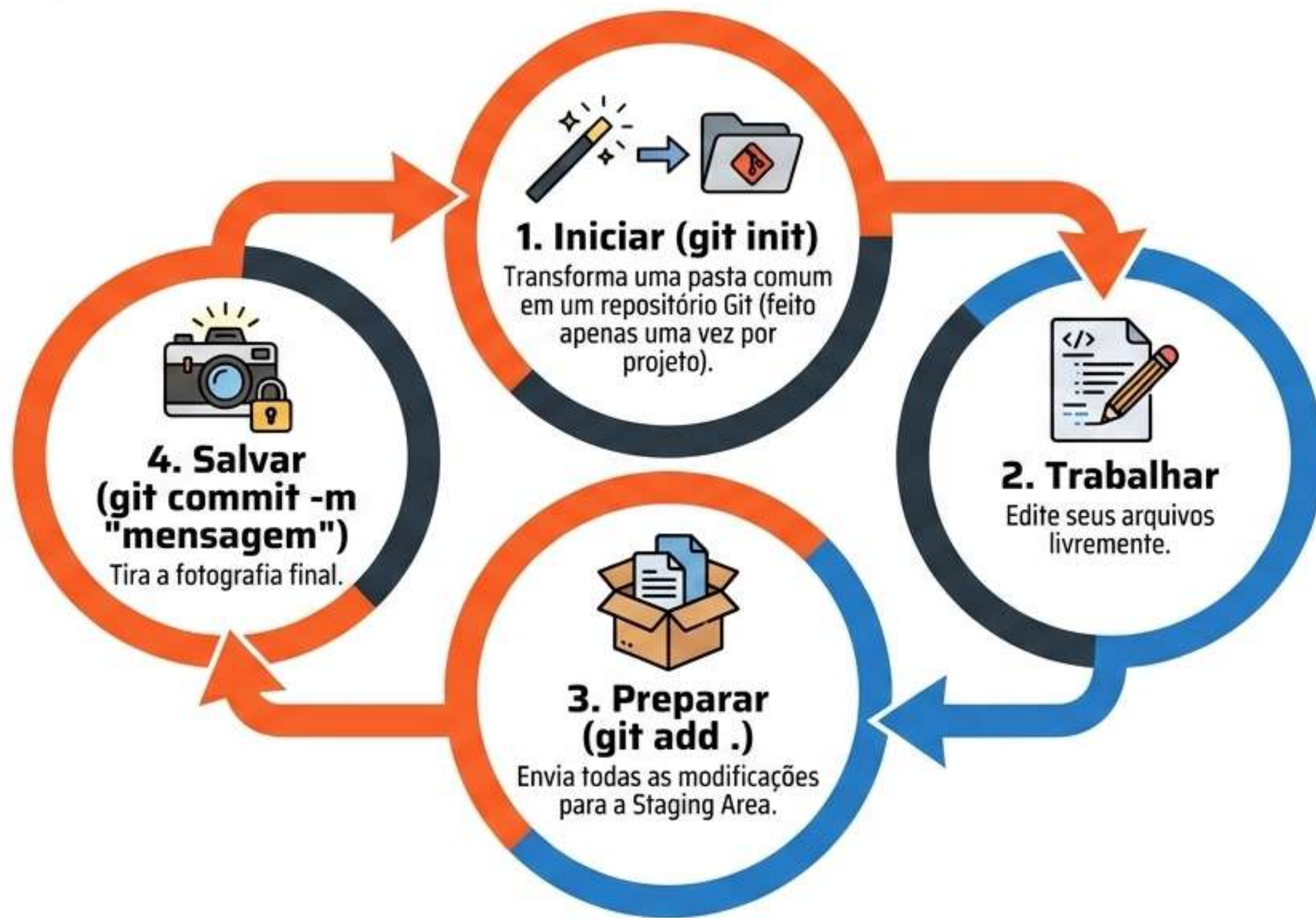
Diga **ao Git quem você é (crucial para trabalhos em equipe)**:

```
> git config --global user.name  
"[Your Name]"
```

```
> git config --global user.email  
"[Your Email]"
```

O parâmetro **--global** garante que essa identidade seja usada em todos os projetos do seu PC.

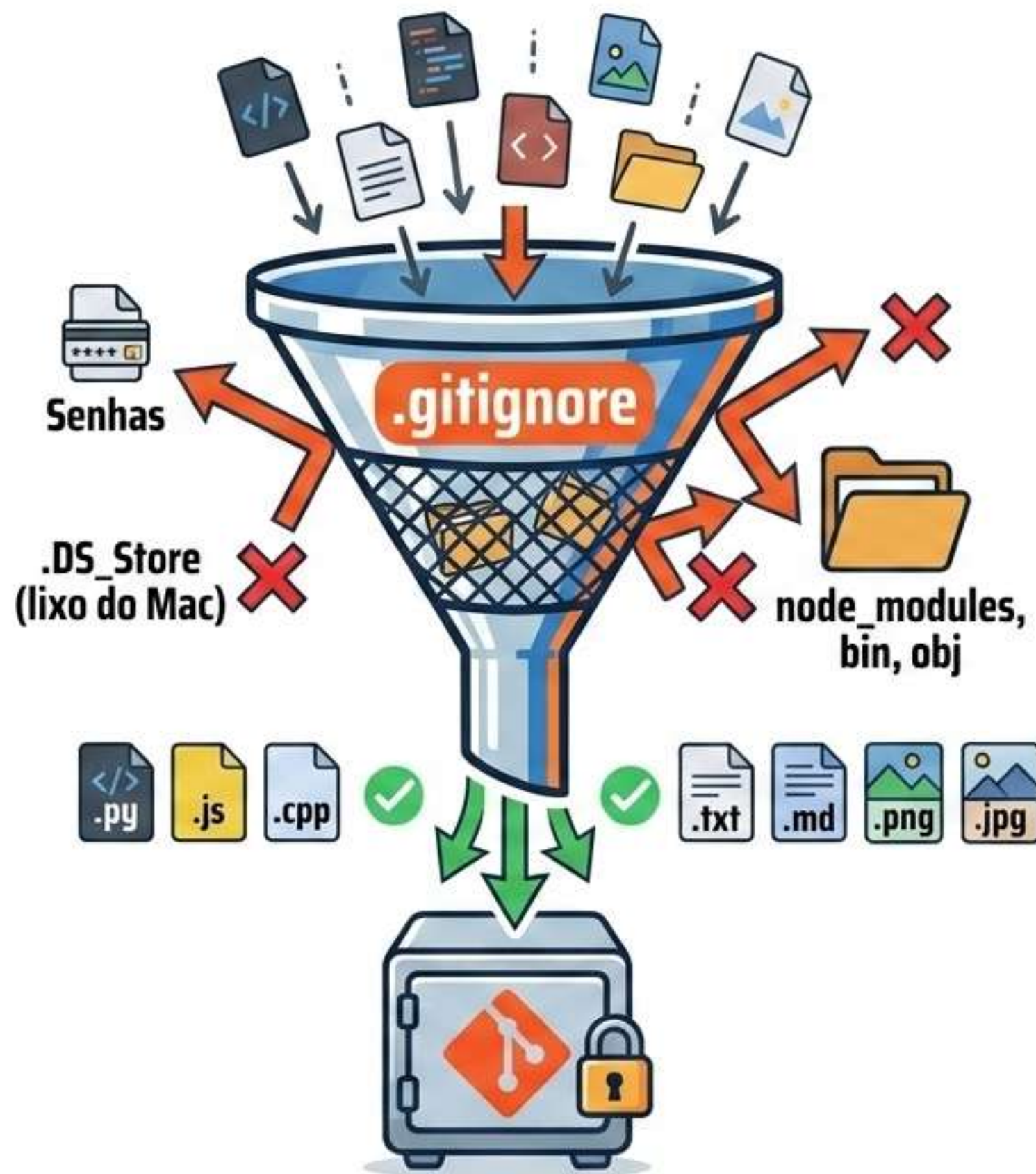
A Estratégia Solo: Sua Rotina Diária



DICA DE OURO:

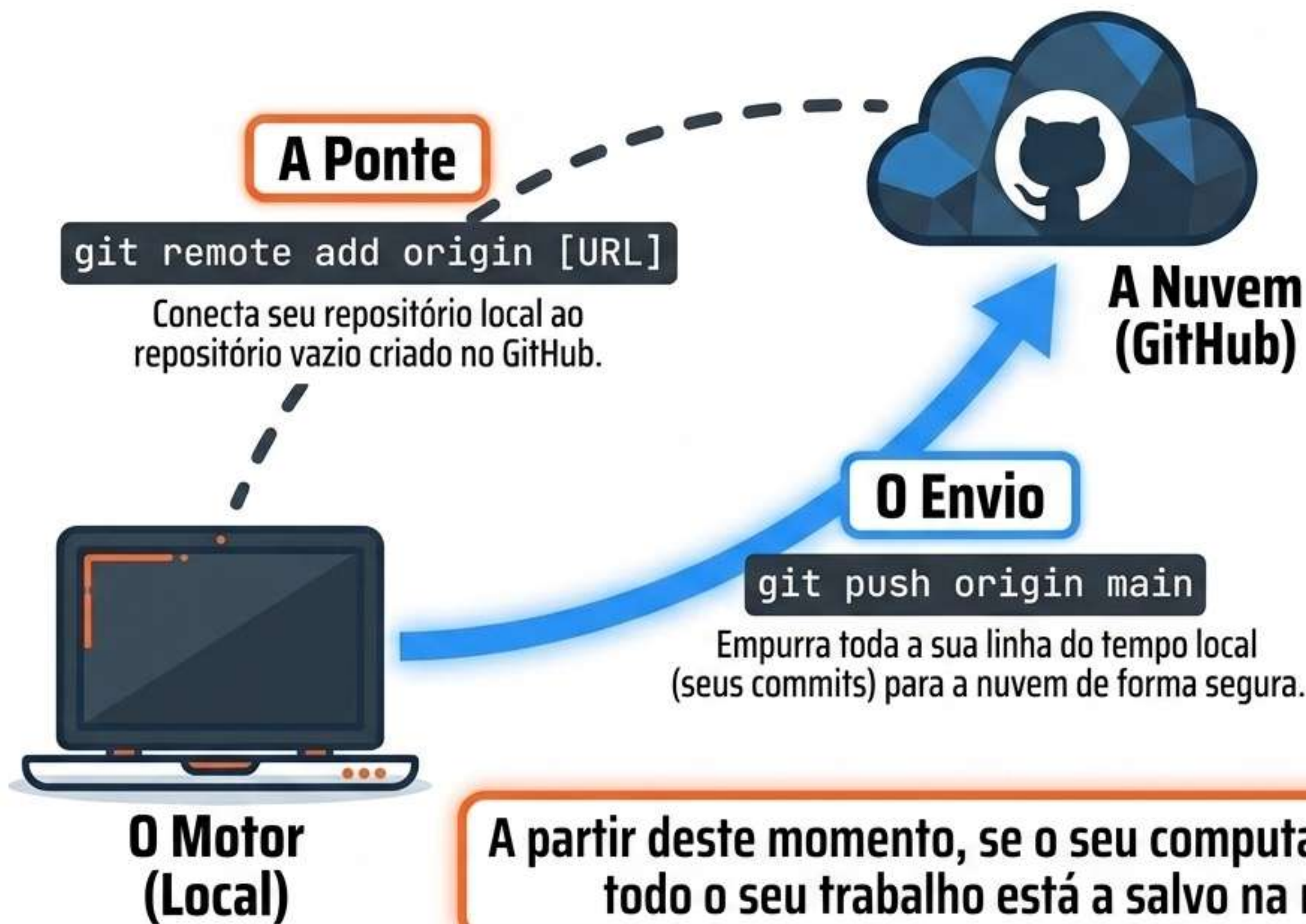
Não espere o trabalho acabar para comitar. Faça commits pequenos a cada nova tarefa concluída!

Mantendo o Lixo Fora (.gitignore)



O arquivo `.gitignore` atua como um segurança de balada. Você lista o que o Git deve ignorar, garantindo que arquivos pesados, irrelevantes ou segretos nunca entrem na sua máquina do tempo.

Indo para a Nuvem: Backup Remoto



Trabalhando em Equipe: Branches (Ramificações)



A Sincronização da Equipe na Prática

1. Baixar (git clone)

Traz o repositório da nuvem para o seu PC.
(Feito apenas uma vez).



Aluno A



GitHub Cloud



2. Atualizar (git pull)

Puxa as novas alterações que seu colega enviou para a nuvem, atualizando sua máquina local.



Aluno B

3. Unir (git merge)

Traz o código da branch do seu colega para a branch principal.



Regras de Ouro para Estudantes Desenvolvedores



Commits contam uma história:

Evite mensagens como "ajuste" ou "agora vai". Use "corrigido erro no botão voltar".



Puxe antes de empurrar:

Sempre rode git pull antes de dar git push para evitar conflitos com o código da equipe.



A main é sagrada:

Nunca teste código quebrado diretamente na branch principal. Use branches!



Cuidado com a pasta .git:

Nunca delete a pasta oculta .git do seu projeto, ou a máquina do tempo será destruída.

Muito Obrigado!



Referências & Agradecimentos:

- "Git & GitHub Essentials" - Rocketseat (YouTube)
- "Git e GitHub do Zero" - Raniery Programador (YouTube)
- Documentação Oficial: git-scm.com